

v1.0.0

OpenAPI 3.1.0

SmartPoolConnect API

SmartPoolConnect API Support



Proprietary

[Download OpenAPI Document](#)

Public API for integrating with the SmartPoolConnect platform.

Use this API to manage and monitor pool equipment — filtration, pH/chlorine dosing, heating, lighting, and more. Supports pool hardware versions v1, v2, and v3.

Authentication

Two authentication methods are supported:

API Key (Recommended for 3rd-party integrations)

Include your API key in the `X-API-Key` header:

```
X-API-Key: spc_your_api_key_here
```

Keys are issued by an administrator. Contact your admin or email api-support@smartpoolconnect.eu to request one. The key inherits the permissions of the user it is bound to (admin / master / pool builder / owner) and further restricts what it can do via its scope list.

OAuth 2.0 Bearer Token

Obtain a JWT token via the OAuth 2.0 token endpoint and include it in the `Authorization` header:

```
Authorization: Bearer <your-access-token>
```

Primarily used by the first-party web UI; new integrations should prefer the API Key flow.

Quickstart — 5 minutes to your first request

Assuming an admin has already issued you a key (otherwise email api-support@smartpoolconnect.eu):

1. List the pools you have access to

```
curl https://api.smartpoolconnect.eu/pool \
-H "X-API-Key: spc_your_key_here"
```

2. Read one pool's current state (pH / Rx / temperature)

```
curl "https://api.smartpoolconnect.eu/pool/$PID" \
-H "X-API-Key: spc_your_key_here"
```

3. Update a setting (V1 / V2 pools)

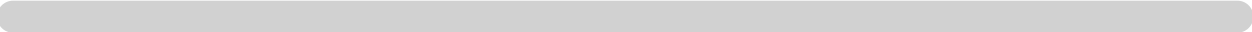
```
curl -X PATCH "https://api.smartpoolconnect.eu/pool/$PID/ph" \
-H "X-API-Key: spc_your_key_here" \
-H "Content-Type: application/json" \
-d '{"target": 7.2}'
```

4. Read a V3 pool table

```
curl "https://api.smartpoolconnect.eu/pool/$PID/v3/temperature_config" \
-H "X-API-Key: spc_your_key_here"
```

5. Queue a V3 setting change

```
curl -X PATCH "https://api.smartpoolconnect.eu/pool/$PID/v3/temperature_config" \
-H "X-API-Key: spc_your_key_here" \
-H "Content-Type: application/json" \
-d '{"setpoint": 28.0}'
```



Python example (V3 batch read)

```
import requests
r = requests.get(
    f"https://api.smartpoolconnect.eu/pool/{PID}/v3",
    headers={"X-API-Key": "spc_your_key_here"},
)
```

```
r.raise_for_status()
tables = r.json() # { "filter_config": { "data": {...} }, ... }
```

JavaScript example (Home Assistant custom integration)

```
const res = await fetch(
  `https://api.smartpoolconnect.eu/pool/${pid}`,
  { headers: { "X-API-Key": process.env.SPC_API_KEY } }
);
const pool = await res.json();
console.log("Water:", pool.temperature.metrics.water_temp, "°C");
```

Available Scopes

API keys can be configured with the following scopes:

- `pools:read` — List and view pool details
- `pools:write` — Create, update, and delete pools
- `pools:claim` — Claim a new pool by MAC address
- `controls:read` — Read pool control settings (pH, Cl, heating, etc.)
- `controls:write` — Modify pool control settings
- `history:read` — Access historical sensor data
- `firmware:upload` — Upload firmware images
- `audit:read` — Read API-key audit logs
- `webhooks:write` — Manage webhook subscriptions (coming soon)

Scope enforcement (since 2026-07-08): scopes are enforced on every request.

Reads (GET) under `/pool` and `/search` require any of `pools:read`, `controls:read`, or `history:read`. Writes (config PATCHes, `/pool/{pid}/cmd/*` commands) require `controls:write` or `pools:write`. Claiming a pool (POST `/pool`) requires `pools:claim` or `pools:write`. A key lacking the required scope receives 403 with body `{ "error": "missing_scope", "required_any_of": [...] }`. Denied calls are recorded in the key's activity log, as are rate-limited (429) calls.

Pool Versions

- **v1** — Legacy hardware. Read-only status; full pool list + history available.
- **v2** — Standard hardware. Full control + configuration (pH, chlorine, filter, heating, lighting, level, aux slots, deck cover).

- **v3** — Next-gen hardware. Use the `/pool/{pid}/v3` and `/pool/{pid}/v3/{table}` endpoints for direct access to the `proc_v3` schema (config + live data). The V1/V2-shaped routes return `Unsupported version` for V3 pools.

Reading sensor + flow data

Sensor readings live in the `metrics` block of each module on `GET /pool/{pid}`. The same shape is returned for V1, V2 and V3 pools — the Rust API translates V3 firmware tables into V1/V2-shaped objects so integrations can have a single code path.

Reading	Where to look (<code>GET /pool/{pid}</code>)	Notes
pH	<code>ph.metrics.actual</code> (float, pH units)	
Redox / Cl mV	<code>cl.metrics.actual</code> (int, mV)	RX-style probes
Free chlorine	<code>cl.metrics.clm</code> (float, ppm)	CLM-style probes
Water temperature	<code>temperature.metrics.water_temp</code> (float, °C)	Populated whenever the probe is wired (does not require heating to be enabled)
Ambient temperature	<code>temperature.metrics.ambient_temp</code> (float, °C)	
Filter pump speed	<code>filter.metrics.pump_speed</code> (int, see the Pump speed codes table) AND <code>filter.status.pump_status</code> (int, see Pump status codes)	<code>> 0</code> ⇒ pump is running
Filter motor current	<code>filter.metrics.pump_current</code> (float, A)	
Dosing channel state	<code>ph.status.status</code> and <code>cl.status.status</code> (int enum)	No-flow signal is version-specific: <code>201</code> ("No Flow") on V1/V2 systems, <code>-28</code> ("Off: No flow") on V3/Nexus 2

How to detect "pump running but no flow" (dry-run risk)

A pool is at risk of running dry when the filter pump is commanded on AND one of the dosing channels reports the no-flow code — `201` on V1/V2 systems, `-28` on V3:

```
pool = requests.get(f"https://api.smartpoolconnect.eu/pool/{pid}", headers=headers)

pump_on = (pool["filter"]["status"]["pump_status"] > 0
            and pool["filter"]["metrics"]["pump_speed"] > 0)
NO_FLOW = (201, -28) # 201 = V1/V2 "No Flow", -28 = V3 "Off: No flow"
no_flow = (pool["ph"]["status"]["status"] in NO_FLOW
            or pool["cl"]["status"]["status"] in NO_FLOW)
dry_run_risk = pump_on and no_flow
```

The no-flow decision is made by the firmware itself based on the pool's configured flow-detection mode (paddle / direct / pressure / modbus pump), so the integration does not need to know the flow-source type.

Alternate path for V3 — raw sensor tables

V3 pools also expose the underlying tables directly, which is sometimes useful when an integration wants the raw firmware fields without the V1/V2 translation:

- `GET /pool/{pid}/v3/filter_data` — live filter fields including `actual` (current speed), `control` (commanded speed), `mb_speed`, `mb_watt`, `mb_status`, `mb_err_flag`, `mb_err_code`
- `GET /pool/{pid}/v3/dosing_data` — live dosing fields including `ph_control` and `cl_control` (both carry the same `-28` "Off: No flow" code documented above)
- `GET /pool/{pid}/v3/analogue_data` — raw probe readings: `water_t`, `ambient_t`, `ph`, `rx`, `tds_pollution`, `tds_salt`, plus `ph_input_error` / `rx_input_error`

Controlling & configuring the pool (v1 / v2)

Every call goes to the cloud at `https://api.smartpoolconnect.eu`. The pool cabinet does **not** host a local API — it only sends its data outbound to the cloud — so there is no endpoint on the pool's own IP address.

Momentary commands — `POST /pool/{pid}/cmd/{command}`

For one-off actions (not persistent settings). No request body; the pool applies the command on its next sync.

command	effect	versions
<code>cover_open</code> / <code>cover_stop</code> / <code>cover_close</code>	open / stop / close the deck cover	v1, v2
<code>backwash</code>	start a backwash cycle	v2
<code>shock_start</code> / <code>shock_stop</code>	start / stop shock chlorination	v2
<code>lighting_next</code> / <code>lighting_reset</code>	next colour / reset colour (RGB & rotation lighting)	v2

```
curl -X POST "https://api.smartpoolconnect.eu/pool/$PID/cmd/cover_open" \
-H "X-API-Key: spc_your_key_here"
```

Settings — `GET` / `PATCH` `/pool/{pid}/{module}`

For persistent configuration: `GET` the current config, then `PATCH` your changes back.

Modules (v1/v2): `ph`, `cl`, `filter`, `cover`, `temperature`, `temperature/zero-e`, `temperature/solar`, `lighting`, `level`, `aux/{slot}`, and `spec` (general settings). V3 pools use `/pool/{pid}/v3/{table}` instead.

How `PATCH` works — two things that trip up most integrations

1. `PATCH` replaces the whole config object — it is NOT a partial merge. Send every field the module has (use a `GET` response as your template), not just the field you changed. A partial body such as `{"schedule_1": {"enabled": true}}` is rejected with `did not match any variant of untagged enum ...`.
2. `GET` wraps the config; `PATCH` does not. A `GET` returns `{"config": { ... }, "status": { ... }, "metrics": { ... }}` — the `PATCH` body is only the inner config object, at the top level. Do not send the `config` / `status` / `metrics` wrapper.

So the reliable recipe is `GET` → edit → `PATCH` the whole object back, un-wrapped.

Example (enable filter schedule 1 — note the top-level fields, and that all three schedules are sent):

```
# read the current config; the PATCH body is exactly the object under "config"
curl "https://api.smartpoolconnect.eu/pool/$PID/filter" -H "X-API-Key: spc_your_key_here"

curl -X PATCH "https://api.smartpoolconnect.eu/pool/$PID/filter" \
-H "X-API-Key: spc_your_key_here" -H "Content-Type: application/json" \
-d '{"always_active": false, "pump_speed": "low",
```

```
"schedule_1": {"enabled": true, "pump_speed": "medium", "start_time": "1  
"schedule_2": { ...unchanged from GET... }, "schedule_3": { ...unchanged
```

- **Filter schedules** — `PATCH /pool/{pid}/filter`. Up to three schedules (`schedule_1/2/3`), each with an on/off flag, start/stop time, days and a `pump_speed`, plus an `always_active` mode with its own `pump_speed`. See the `FilterConfig` schema on that endpoint for the exact fields.
- **Cover behaviour** — `PATCH /pool/{pid}/cover`. The pump's reaction to the cover lives here (e.g. drop the pump to low speed when the cover closes). There is **no** built-in "run at medium while the cover is open" setting — see the recipe below.
- **Pause the controller** — `PATCH /pool/{pid}/spec` with `{"pause": true}` pauses the whole pool controller (identical to the external OFF-contact / the pause shown top-left on the cabinet display); `{"pause": false}` resumes. The external OFF-contact is also configured on this endpoint.
- **Lighting on/off** — `PATCH /pool/{pid}/lighting` with `{"always_active": true}` (on) or `{"always_active": false}` (off). This is the on/off switch; the `lighting_next` / `lighting_reset` *commands* only cycle RGB colour and do **not** turn the light on/off.

Recipe: run the filter at medium whenever the cover is open

There is no native setting for this, but it is a few lines against the API: poll `GET /pool/{pid}`, read the cover state from the `cover` module, and while the cover is open `PATCH /pool/{pid}/filter` to enable a schedule with `pump_speed = 2` (Medium); switch it back off when the cover closes.

Pump speed & status codes

`pump_speed` — the commanded / current filter speed (`filter.metrics.pump_speed`, and the `pump_speed` fields in `FilterConfig`):

value	meaning
-1	Disabled
0	Off
1	Low
2	Medium
3	High

value	meaning
4	Max

`filter.status.pump_status` — what the pump is currently doing (`filter.status.pump_status`):

value	meaning
-1	Disabled
0	Off
1	Schedule 1
2	Schedule 2
3	Schedule 3
4	Heating
5	Solar
6	Backwash
7	Cooling
8	Manual
9	Frost
10	Active
11	Deck (cover-driven)
12	Valve error
13	Deck low (pump forced to low speed while the cover is closed — normal)
14	Too hot
15	Invalid

Rate Limits

API calls are rate-limited per API key. Default: 60 requests/minute. Rate limit headers are included in every response:

- `X-RateLimit-Limit` — Maximum requests per window

- `X-RateLimit-Remaining` — Remaining requests in current window
- `X-RateLimit-Reset` — Unix timestamp when the window resets

Contact support for increased quotas.

Support

For API key requests or technical support: api-support@smartpoolconnect.eu

Server	
https://api.smartpoolconnect.eu ▾	
Production	

Authentication Required	ApiKeyAuth ▾
API key for 3rd-party integrations. Obtain a key from your administrator. Include the key in... More	
Name: X-API-Key	✕
Value: QUxMIFIPVVlgQkFTRSBBUkUgQkVMT05HIFRPIFVT	👁

Client Libraries					
▸ Shell	Ruby	Node.js	PHP	Python	⋮ More
Shell Curl					

Pools

Pool CRUD operations — list, show, claim, update, and delete pools

Operations
<pre> GET /pool POST /pool GET /pool/{pid} PATCH /pool/{pid} DELETE /pool/{pid} PUT /pool/{pid}/info/avatar </pre>

List pools (with optional MAC / name filter)

Returns a paginated list of pools the authenticated user has access to.

Admin users see all pools. Non-admin users only see pools linked to their tenant or user relations.

Looking up a pool by MAC address

Pass `mac=<address>` to get a single pool back (case-insensitive exact match against `mac_address`, which is unique). This is the recommended way for third-party integrations to map a known device MAC to its pool `pid`:

```
curl "https://api.smartpoolconnect.eu/pool?mac=00:14:2d:ac:9a:a7" \  
-H "X-API-Key: spc_your_key_here"
```

For substring / fuzzy lookups against name OR MAC use `search=<fragment>` instead.

Each item in the response includes both `pid` and `mac_address`, so you can build a lookup map client-side from a single `/pool` call.

GET /pool

Shell Curl ▾

```
1 curl https://api.smartpoolconnect.eu/pool \  
2 --header 'X-API-Key: YOUR_SECRET_TOKEN'
```

▶ Test Request

200 401

Show Schema ☐

```
{  
  "items": [  
    {  
      "pid": "123e4567-e89b-12d3-a456-426614174000",  
      "name": "Olympic Amsterdam",  
      "status": "online",  
      "mac_address": "02:00:00:00:03:01",  
      "created_at": "2026-09-07T15:07:40.417Z",  
      "activity_at": "2026-09-07T15:07:40.417Z",  
      "avatar": "string",  
      "location": null  
    }  
  ]  
}
```

Array of pool objects

Query Parameters

mac string Example

Exact case-insensitive match against `mac_address`. Accepts the canonical colon-separated form (e.g. `00:14:2d:ac:9a:a7`). Because `mac_address` is unique, the response contains at most one pool.

search string

Free-text fragment matched against `pool.name` OR `pool.mac_address` using case-insensitive `ILIKE`. Use for partial matches; prefer `mac` for exact lookup.

page integer · min: 0

Zero-based page index (default 0).

page_size integer · min: 1 · max: 5000

Items per page (default 25, max 5000).

Responses

› **200** Array of pool objects

401 Unauthorized — invalid or expired token

🔑 Auth Required

Claim a pool by MAC address

Register a new pool device by providing its MAC address. The pool must be broadcasting on the network and not already claimed.

POST /pool

Shell Curl ▾

```
1 curl https://api.smartpoolconnect.eu/pool \
2   --request POST \
3   --header 'Content-Type: application/json' \
4   --header 'X-API-Key: YOUR_SECRET_TOKEN' \
5   --data '{
6     "mac_address": "02:00:00:00:03:01"
7   }'
```

▶ Test Request

201 400 409
No Body
Pool successfully claimed

Body

· [ClaimReq](#)

required

application/json

Request to claim a pool device

mac_address string required [Example](#)
MAC address of the pool device to claim

Responses

- 201 Pool successfully claimed
- 400 Invalid MAC address format
- 409 Pool already claimed by another user

🔑 Auth Required

Get pool details

Returns full pool details including all module configurations and live sensor data.

The response structure varies by pool hardware version:

- v2: Returns pH, chlorine, filter, heating, lighting, level, aux slot modules
- v3: Returns extended sensor modules with advanced dosing data
- v1: Returns `501 Not Supported` (use pool list for v1 status)

GET /pool/{pid}Shell Curl ▾

1 curl https://api.smartpoolconnect.eu/pool/123e4567-e89b-12d3-a456-4266141740

2 --header 'X-API-Key: YOUR_SECRET_TOKEN'

▶ Test Request

200 404 501	Show Schema ☐
null	
Pool details with module data	

Path Parameters

pid string · uuid required
Pool UUID

Responses

- > 200 Pool details with module data
- 404 Pool not found or not accessible
- 501 Pool version not supported for detail view (v1)

🔒 Auth Required

Update pool info

Update the pool name or other editable properties.

PATCH /pool/{pid}Shell Curl ▾

```
1 curl https://api.smartpoolconnect.eu/pool/123e4567-e89b-12d3-a456-4266141740
2 --request PATCH \
3 --header 'Content-Type: application/json' \
4 --header 'X-API-Key: YOUR_SECRET_TOKEN' \
5 --data '{
6   "name": "My Pool"
7 }'
```

▶ Test Request

200 404

No Body

Pool updated successfully

Path Parameters

pid string · uuid **required**

Pool UUID

Body · [EditReq](#)

required

application/json

Request to update pool properties

name string **required** Example

New pool display name

Responses

200 Pool updated successfully

404 Pool not found

🔑 Auth Required

Delete (unclaim) a pool

Remove a pool from your account. The physical device is not affected.

DELETE /pool/{pid}

Shell Curl ▾

```
1 curl https://api.smartpoolconnect.eu/pool/123e4567-e89b-12d3-a456-4266141740
2 --request DELETE \
3 --header 'X-API-Key: YOUR_SECRET_TOKEN'
```

▶ Test Request

200 404

No Body

Pool removed

Path Parameters

pid string · uuid **required**

Pool UUID

Responses

200 Pool removed

404 Pool not found

🔒 Auth Required

Upload pool avatar image

Upload a custom image for a pool. Accepts common image formats.

PUT /pool/{pid}/info/avatar

Shell Curl ▾

```
1 curl https://api.smartpoolconnect.eu/pool/123e4567-e89b-12d3-a456-4266141740
2 --request PUT \
3 --header 'X-API-Key: YOUR_SECRET_TOKEN'
```

▶ Test Request

200

No Body

Avatar uploaded successfully

Path Parameters

pid string · uuid required

Pool UUID

Responses

200 Avatar uploaded successfully

Firmware

Firmware version management for pool hardware

Operations

POST /pool/{pid}/firmware

Auth Required

Update pool firmware versions

Report firmware version information for a pool device.

POST /pool/{pid}/firmware

Shell Curl ▾

1 curl https://api.smartpoolconnect.eu/pool/123e4567-e89b-12d3-a456-4266141740

2 --request POST \

3 --header 'Content-Type: application/x-www-form-urlencoded' \

4 --header 'X-API-Key: YOUR_SECRET_TOKEN' \

5 --data 'null'

► Test Request

200

No Body

Firmware versions updated

Path Parameters

pid string · uuid **required**
Pool UUID

Body · PoolFirmwareReq **required** application/x-www-form-urlencoded

Responses

200 Firmware versions updated

Search

Full-text search across pools and devices

Operations
GET /search

 Auth Required

Search pools and devices

Full-text search across pool names, MAC addresses, and other identifiers.

GET /search Shell Curl ▾

```
1 curl 'https://api.smartpoolconnect.eu/search?query=' \  
2 --header 'X-API-Key: YOUR_SECRET_TOKEN'
```

▶ Test Request

200

Show Schema ☐

```
[
  {
    "type": "string",
    "id": "123e4567-e89b-12d3-a456-426614174000",
    "name": "string",
    "confidence": 1,
    "description": "string"
  }
]
```

Search results

Query Parameters

query string **required**

Search query string

Responses

> **200** Search results

Controls

Send commands and read/update pool configuration — cover, filter, lighting, general settings + pause

Operations

```
POST /pool/{pid}/cmd/{command}
GET /pool/{pid}/filter
PATCH /pool/{pid}/filter
GET /pool/{pid}/cover
PATCH /pool/{pid}/cover
GET /pool/{pid}/lighting
PATCH /pool/{pid}/lighting
GET /pool/{pid}/spec
PATCH /pool/{pid}/spec
```

🔒 Auth Required

Send a momentary command (cover, backwash, shock, lighting)

Queue a one-off command for a v1 / v2 pool. No request body; the pool applies it on its next sync.

`command` is one of: `cover_open`, `cover_stop`, `cover_close` (v1 & v2); `backwash`, `shock_start`, `shock_stop`, `lighting_next`, `lighting_reset` (v2). V3 pools return `Unsupported version`.

Lighting on/off is not a command. `lighting_next` / `lighting_reset` only cycle the RGB colour. To switch lighting on or off, use `PATCH /pool/{pid}/lighting` with `{"always_active": true|false}`.

For persistent cover *behaviour* (not opening/closing) use `PATCH /pool/{pid}/cover`; to pause the whole controller use `PATCH /pool/{pid}/spec`.

POST /pool/{pid}/cmd/{command} Shell Curl ▾

```
1 curl https://api.smartpoolconnect.eu/pool/123e4567-e89b-12d3-a456-4266141740
2 --request POST \
3 --header 'X-API-Key: YOUR_SECRET_TOKEN'
```

▶ Test Request

200 404

No Body

Command queued

Path Parameters

pid string · uuid **required**

Pool UUID

command string · enum **required**

Command to send

values

- cover_open
- cover_stop
- cover_close
- backwash
- shock_start
- shock_stop
- lighting_next
- lighting_reset

Responses

200 Command queued

404 Pool not found

🔒 Auth Required

Get filter configuration

Filter pump schedules and speeds (v1 / v2).

GET /pool/{pid}/filter

Shell Curl ▾

1 curl https://api.smartpoolconnect.eu/pool/123e4567-e89b-12d3-a456-4266141740

2 --header 'X-API-Key: YOUR_SECRET_TOKEN'

▶ Test Request

200

Show Schema ☐

null

Filter config

Path Parameters

pid string · uuid required

Pool UUID

Responses

> 200 Filter config

🔒 Auth Required

Update filter configuration

Set up to three schedules (`schedule_1/2/3`) — each with on/off, start/stop time, days and `pump_speed` (0=Off ... 4=Max) — plus an `always_active` mode with its own `pump_speed`.

To run the filter at medium while the cover is open (there is no built-in setting), read the cover state from `GET /pool/{pid}` and enable a schedule with `pump_speed: 2` while the cover is open.

PATCH /pool/{pid}/filter

Shell Curl ▾

1 curl https://api.smartpoolconnect.eu/pool/123e4567-e89b-12d3-a456-4266141740

2 --request PATCH \

3 --header 'Content-Type: application/json' \

4 --header 'X-API-Key: YOUR_SECRET_TOKEN' \

5 --data 'null'

▶ Test Request

200 404

No Body

Filter updated

Path Parameters

pid string · uuid required

Pool UUID

Responses

200 Filter updated

404 Pool not found

Auth Required

Get cover configuration

Deck cover behaviour (v1 / v2), including how the filter pump reacts to the cover. To open/stop/close the cover, use `POST /pool/{pid}/cmd/{command}`.

GET /pool/{pid}/cover

Shell Curl ▾

1 curl https://api.smartpoolconnect.eu/pool/123e4567-e89b-12d3-a456-4266141740

2 --header 'X-API-Key: YOUR_SECRET_TOKEN'

▶ Test Request

200

Show Schema ☐

null

Cover config

Path Parameters

pid string · uuid required
Pool UUID

Responses

> 200 Cover config

Update cover configuration

Configure the cover — e.g. drop the pump to low speed when the cover closes. This does NOT move the cover; to open/stop/close it, use `POST /pool/{pid}/cmd/cover_open|cover_stop|cover_close`.

PATCH /pool/{pid}/cover

Shell Curl ▾

1 curl https://api.smartpoolconnect.eu/pool/123e4567-e89b-12d3-a456-4266141740

2 --request PATCH \

3 --header 'Content-Type: application/json' \

4 --header 'X-API-Key: YOUR_SECRET_TOKEN' \

5 --data 'null'

▶ Test Request

200 404
No Body
Cover updated

Path Parameters

pid string · uuid required
Pool UUID

Body · CoverConfig required application/json

Responses

200	Cover updated
404	Pool not found

Get lighting configuration

Current lighting configuration (v1 / v2): on/off (`always_active`), type, colour, dimming and schedule. V3 pools return `Unsupported version`.

`GET /pool/{pid}/lighting`

Shell Curl ▾

```
1 curl https://api.smartpoolconnect.eu/pool/123e4567-e89b-12d3-a456-4266141740
2 --header 'X-API-Key: YOUR_SECRET_TOKEN'
```

▶ Test Request

200

Show Schema ☐

null

Lighting config

Path Parameters

pid string · uuid required
Pool UUID

Responses

▾ 200

application/json

Lighting config

🔒 Auth Required

Turn lighting on/off (or update its configuration)

Switch the pool lighting **on or off**, or update its full configuration (v1 / v2).

On / off — send just `{"always_active": true}` to turn the lights on, or `{"always_active": false}` to turn them off (if a time schedule is configured, `false` hands control back to that schedule):

```
curl -X PATCH "https://api.smartpoolconnect.eu/pool/$PID/lighting" \
  -H "X-API-Key: spc_your_key_here" -H "Content-Type: application/json" \
  -d '{"always_active": true}'
```

Lighting on/off is a **configuration** value, not a momentary command — `POST /pool/{pid}/cmd` does not control it. The `lighting_next` / `lighting_reset` commands there only cycle the **RGB colour** (and do nothing on a single-colour lamp).

Full config — send the complete object (`type`, colour, dimming, schedule, ...) exactly as returned by `GET /pool/{pid}/lighting`.

PATCH /pool/{pid}/lighting Shell Curl ▾

```
1 curl https://api.smartpoolconnect.eu/pool/123e4567-e89b-12d3-a456-4266141740
2 --request PATCH \
3 --header 'Content-Type: application/json' \
4 --header 'X-API-Key: YOUR_SECRET_TOKEN' \
5 --data '{
6   "always_active": true
7 }'
```

▶ Test Request

200 404

No Body

Lighting updated

Path Parameters

pid string · uuid **required**

Pool UUID

Body

required

application/json

One of On/off switch ▾

always_active boolean **required**

`true` = lighting always on; `false` = off / follow the time schedule.

Responses

200 Lighting updated

404 Pool not found

🔒 Auth Required

Get general pool settings

General controller settings (v1 / v2): `pause`, the external OFF-contact, which sensors are present, deck/heating capabilities, pool volume, etc.

GET /pool/{pid}/spec

Shell Curl ▾

1 curl https://api.smartpoolconnect.eu/pool/123e4567-e89b-12d3-a456-4266141740

2 --header 'X-API-Key: YOUR_SECRET_TOKEN'

▶ Test Request

200

Show Schema ☐

null

Spec config

Path Parameters

pid string · uuid **required**
Pool UUID

Responses

▾ 200

application/json

Spec config

🔒 Auth Required

Update general pool settings (incl. pause)

PATCH `{"pause": true}` to pause the whole controller — the same effect as the external OFF-contact / the pause shown top-left on the cabinet display — and `{"pause": false}` to resume.

PATCH /pool/{pid}/spec

Shell Curl ▾

1 curl https://api.smartpoolconnect.eu/pool/123e4567-e89b-12d3-a456-4266141740

2 --request PATCH \

3 --header 'Content-Type: application/json' \

4 --header 'X-API-Key: YOUR_SECRET_TOKEN' \

5 --data 'null'

▶ Test Request

200 404
No Body
Settings updated

Path Parameters

pid string · uuid required
Pool UUID

Body · SpecConfig required application/json

Responses

- 200 Settings updated
- 404 Pool not found

Models

▾ Pool

A pool device registered on the platform

activity_at string · date-time required

Last communication from the device

created_at string · date-time **required**

When the pool was claimed

mac_address string **required** Example

Hardware MAC address

name string **required** Example

User-assigned pool name

pid string · uuid **required**

Unique pool identifier

status string · Status · enum **required**

Current device connectivity status

values

online

offline

maintenance

avatar string · Blob · nullable

Optional pool image

location nullable

GPS coordinates

▼ Status

string · enum

Current device connectivity status

values

online

offline

maintenance

▼ Blob

string

Base64-encoded binary data (images, etc.)

▼ ClaimReq

Request to claim a pool device

mac_address string **required** Example

MAC address of the pool device to claim

▼ EditReq

Request to update pool properties

name string **required** Example
New pool display name

▼ Search

Search result entry

confidence number · float **required**
Match confidence score (0-1)

description string **required**

id string · uuid **required**

name string **required**

type string **required**
Result type (e.g. "pool")

▼ Location

▼ PoolModule

▼ FilterConfig

▼ CoverConfig

▼ LightingConfig

▼ SpecConfig

▼ PoolFirmwareReq